

# CS 4032 – Web Programming





# React



# React

- A JavaScript library created by Jordan Walke at Facebook
- React allows creation of reusable UI components or user interfaces.
- It is an open-source, component-based front end library which is responsible only for the view layer of the application
- React is used to build single page applications.
- Moved beyond the web into other environments like mobile apps and VR
- Spawned a host of other libraries and frameworks known as the React Stack or React and Friends
- Latest Version of React.JS is V18 (March 2022).



# React is "V" in "MVC"

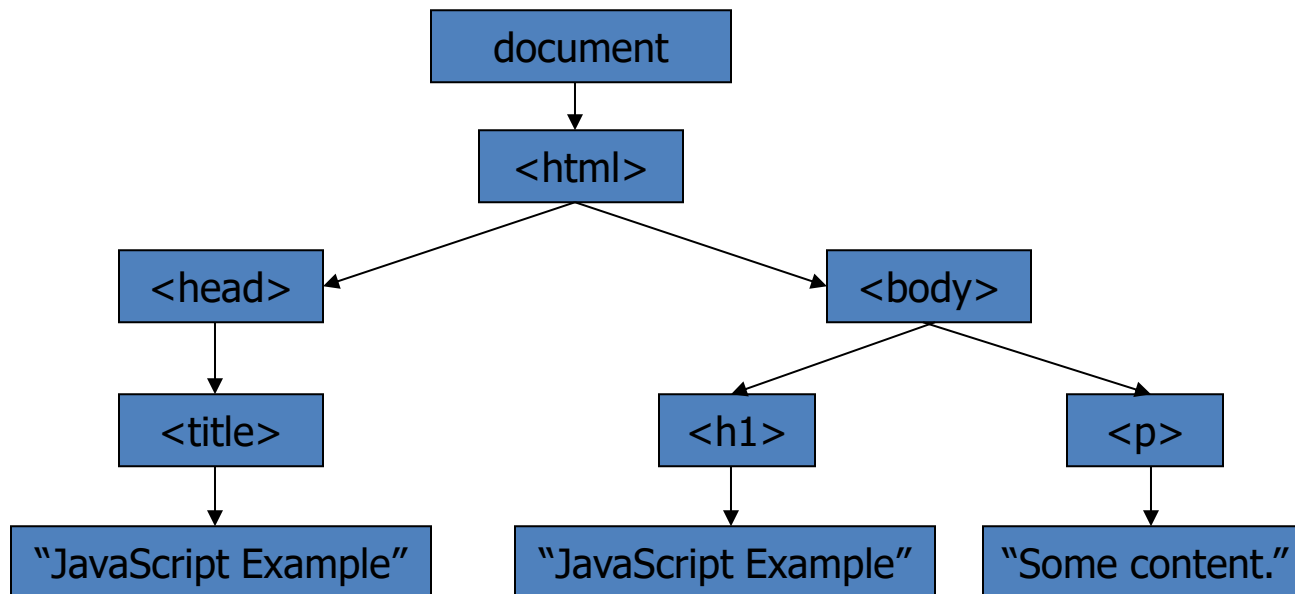
- React is often used as the View in a Model View Controller architecture.
- It provide an efficient way to update the DOM in response to **state** changes
- As it only re-renders things that have changed, rather than just re-rendering everything

# Manipulate An HTML Document

```
<html>
<head><title>JavaScript Example</title></head>
<body>
  <h1>JavaScript Example</h1>
  <p>Some content.</p>
</body>
</html>
```

- As a text file – very difficult
- *As an object*

# Document Object Model (DOM)



Common terminology: parent, child, sibling, ancestor, descendant

# How does React Work

- React creates a VIRTUAL DOM in memory.
- Instead of manipulating the browser's DOM directly, React **creates a virtual DOM in memory**, where it **does all the necessary manipulating**, before making the changes in the browser DOM.
- React finds out what changes have been made, and change **only** what needs to be changed.
- *A reconciliation* process reflects the changes in virtual DOM in the real DOM

# Why React JS

- The main objective of React is to develop User Interfaces (UI) that improves the speed of the apps.
- It uses virtual DOM (JavaScript object), which improves the performance of the app.
- The JavaScript virtual DOM is faster than the regular DOM.
- We can use React JS on the client and server-side as well as with other frameworks.
- It uses component and data patterns that improve readability and helps to maintain larger apps.





# React Features

The important features of React are as following.

- **JSX** - JavaScript Extension (Its an XML or HTML like syntax)
- **Components**
- **One-way Data Binding**
- **Virtual DOM**
- **Simplicity**
- **Performance**



# Technical requirements

*Once Node.js is installed, you can run below command to install react*

***npm install -g create-react-app***

- *To create a new react App, run below command*

***npx create-react-app test\_app***

*After successful creation of react app, below commands will be displayed.*

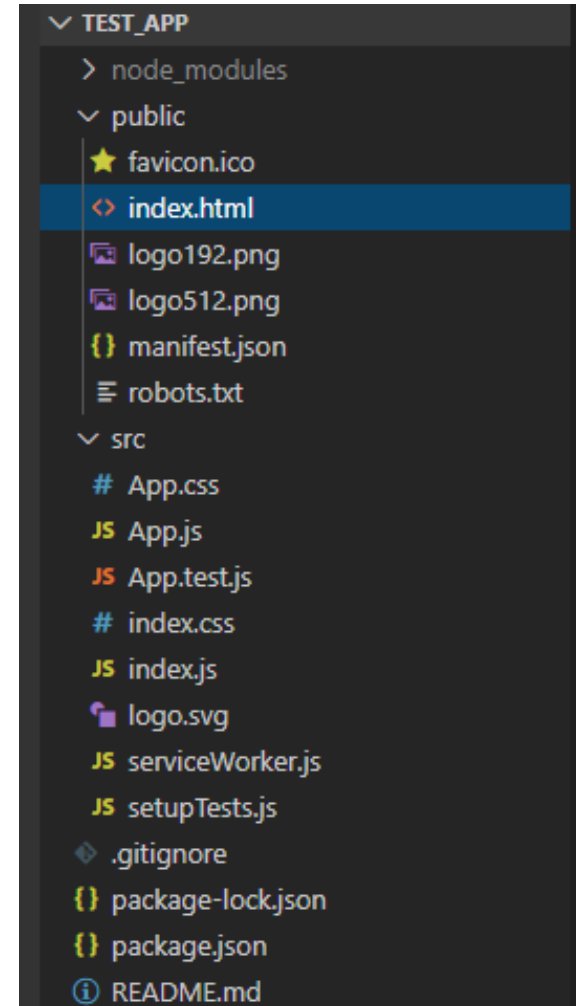
- npm start - This command will start the development server.
- npm run build - Bundles the app into static files for production.
- npm test - starts the test runner.
- npm run eject - Removes this tool and copies build dependencies, configuration files, and scripts into the app directory. If you do this, you can't go back!
- Now you need to start the development server for react by using command

**npm start**

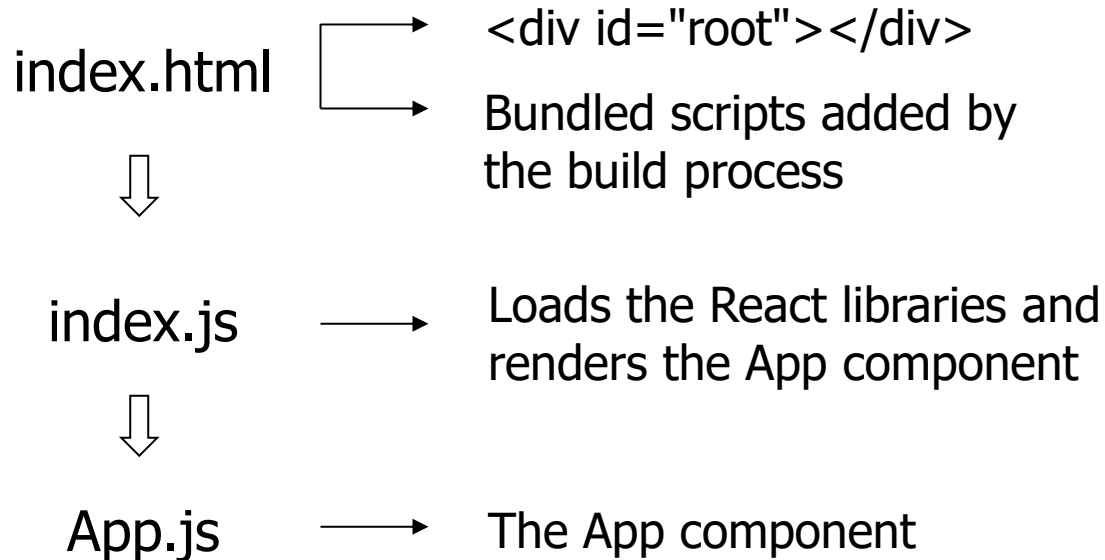


# Folder Structure

- For the project to build, these files must exist with exact filenames:
- **public/index.html** is the page template;
- **src/index.js** is the JavaScript entry point.



# Basic App Structure



# React Directly in HTML

- The quickest way start learning React is to write React directly in your HTML files.

```
<html>
  <script src="https://unpkg.com/react@16/umd/react.production.min.js"></script>
  <script src="https://unpkg.com/react-dom@16/umd/react-dom.production.min.js"></script>
  <script src="https://unpkg.com/babel-standalone@6.15.0/babel.min.js"></script>
  <body>

    <div id="mydiv"></div>

    <script type="text/babel">
      class Hello extends React.Component {
        render() {
          return <h1>Hello World!</h1>
        }
      }
      ReactDOM.render(<Hello />, document.getElementById('mydiv'))
    </script>
  </body>
</html>
```

# Render HTML - React v17

- React's goal is in many ways to render HTML in a web page.
- React renders HTML to the web page by using a function called ReactDOM.render().
- Example
  - Display a paragraph inside the "root" element:

```
ReactDOM.render(<p>Hello</p>, document.getElementById('root'));
```

Index.js

```
<body>  
  <div id="root"></div>  
</body>
```

Index.html



# Render HTML - React v18

- In *Index.js*, update **ReactDOM.render** to **ReactDOM.createRoot** to create a root, and render your app using root.
- Example
  - Display a paragraph inside the "root" element:

```
const container = document.getElementById('root');  
  
// create a root  
const root = ReactDOM.createRoot(container);  
  
//render app to root  
root.render(<p>Hello</p>);
```

Index.js

```
<body>  
  <div id="root"></div>  
</body>
```

Index.html

# JSX

```
var myDiv = (  
  <div class="jsx">  
    Greetings from JSX world!  
  </div>  
);  
ReactDOM.render(myDiv, getElementById('container'));
```

- JSX => JavaScript XML.
- JSX makes it easier to write and add HTML in React.
- JSX is an extension of the JavaScript language based on ES6, and is translated into regular JavaScript at runtime.
- JSX converts HTML tags into react elements.
- JSX uses HTML syntax to create elements and components. It is just like a templating syntax. It has the **tag name**, **attributes**, and **children**.
- JSX compiles the code into pure JavaScript which can be understood by the browser.



# JSX- Example

```
const myelement = <h1>I Love JSX!</h1>;  
ReactDOM.render(myelement, document.getElementById('root'));
```

## Expressions in JSX

With JSX you can write expressions inside curly braces { }.

The expression can be a **React variable, or property, or any other valid JavaScript expression**. JSX will execute the expression and return the result:

```
const myelement = <h1>React is {5 + 5} times better with JSX</h1>;  
ReactDOM.render(myelement, document.getElementById('root'));
```

**React is 10 times better with JSX**

# JSX – Display List

To write HTML on multiple lines, put the HTML inside parentheses:

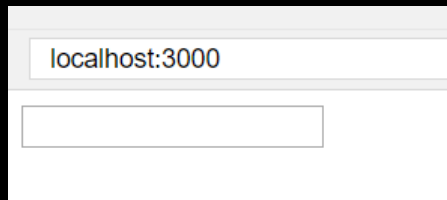
```
const myelement = (  
  <ul>  
    <li>Apples</li>  
    <li>Bananas</li>  
    <li>Cherries</li>  
  </ul>  
);
```

```
ReactDOM.render(myelement, document.getElementById('root'));
```

# JSX –Display an input field

```
const myelement = <input type="text" />;
```

```
ReactDOM.render(myelement,  
document.getElementById('root'));
```



# React Components

- Components let you split the UI into independent, reusable pieces, and think about each piece in isolation.
- They serve the same purpose as JavaScript functions, but work in isolation and returns HTML via a render function.
- Conceptually, components are like JavaScript functions. They accept arbitrary inputs (called “props”) and return React elements describing what should appear on the screen.
- Example -

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```



# Props

- React allows to pass information/Data to a Component using something called **props** (stands for properties).
- Props are arguments passed into React components via HTML attributes.
- React Props are like **function arguments in JavaScript and attributes in HTML**.
- Props flow downwards from the parent component this is why people refer to **React as having *uni-directional* data flow**.
  - In other words. Components receive data from the parent
- Props are immutable, i.e., the value for properties cannot be changed.
- It gives a way to pass data from one component to other components.

# Props -Example

```
import React from "react";
import ReactDOM from "react-dom";

function App() {
  return <Greeting name="Hamza" age={25} occupation="Software Developer" />;
}

function Greeting(props) {
  return (
    <p>
      Hello! I'm {props.name}, a {props.age} years old {props.occupation}.
      Pleased to meet you!
    </p>
  );
}

ReactDOM.render(<App />, document.getElementById("root"));
```

# Component Types

There are 2 types of components in React.js

- Functional Component
- Class Component

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}  
  
const element = <Welcome name="Sara" />;  
ReactDOM.render(  
  element,  
  document.getElementById('root')  
);
```

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

# Functional Component

Functional component includes simple JavaScript functions which takes **props** as an argument and returns a react element to be rendered on the page

- => **Functional** because they are basically functions
- => Can be **Stateless or Stateful**
- => **Presentational** because all they do is output UI elements





# Functional Component Examples

For example, this code renders "Hello, Sara" on the page:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}  
  
const element = <Welcome name="Sara" />;  
ReactDOM.render(  
  element,  
  document.getElementById('root')  
);
```

# Class Component

- Class components are created using ES6's class syntax.
- They are more complex than functional components including **constructors, life-cycle methods, render( ) function and state (data) management**.
- => **Class** because they are basically classes
- => **Smart** because they can contain logic
- => **Stateful** because they can hold and/or manage local state
- => **Container** because they usually hold/contain numerous other (mostly functional) components

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

# Class Component Example

```
import React, { Component } from 'react';

class ExampleComponent extends Component {
  render() {
    return (
      <div>This is an example component.</div>
    );
  }
}

export default ExampleComponent;
```

# Class Component with Props -Example

## App.js

```
import React, { Component } from 'react';
class App extends React.Component {
  render() {
    return (
      <div>
        <h1> Welcome to { this.props.name } </h1>
      </div>
    );
  }
}
export default App;
```

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App.js';

ReactDOM.render(
  <App name = "WebProgrammingt!!" />,
  document.getElementById('app'));
```



# Class Component with Constructor

- If component has a constructor, the props should always be passed to the constructor and also to the React. Component via the `super()` method.

```
class Car extends React.Component {  
  constructor(props) {  
    super(props);  
  }  
  render() {  
    return <h2>I am a Car!</h2>;  
  }  
}
```

```
ReactDOM.render(myelement, document.getElementById('root'));
```

# How do I choose which component type to use?

- Use a class component if you:
- => need to manage local state
- => need to add lifecycle methods to your component
- => need to add logic for event handlers
- => want to use lifecycle methods of Component

# State

- React has another special built-in object called state
- Allows components to create and manage their own data. So unlike props, components cannot pass data with state, but they can create and manage their state internally.
- A change in the state happens based on user-input, triggering an event
- React components (with state) are rendered based on the data in the state. State holds the initial information.

# Change in State

- When state changes, React gets informed and immediately re-renders the DOM – **not the whole DOM, but only the component with the updated state**. This is one of the reasons why React is fast.
- React get notified on a state change with **setState( )** method.
- The **setState( )** method triggers the re-rendering process for the updated parts. React gets informed, knows which part(s) to change, and does it quickly without re-rendering the whole DOM.



# State Example

```
class Test extends React.Component {  
  constructor() {  
    this.state = {  
      id: 1,  
      name: "test"  
    };  
  }  
  
  render() {  
    return (  
      <div>  
        <p>{this.state.id}</p>  
        <p>{this.state.name}</p>  
      </div>  
    );  
  }  
}
```

# State Hook

- A Hook is a special function that lets you “hook into” React features.
  - For example, **useState()** is a Hook that lets you add React state to function components.
- If you write a function component and realize you need to add some state to it, previously you had to convert it to a class. Now you can use a Hook inside the existing function component.

```
import React, { useState } from 'react';

function Example() {
  // Declare a new state variable, which we'll call "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

```
class Example extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0
    };
  }

  render() {
    return (
      <div>
        <p>You clicked {this.state.count} times</p>
        <button onClick={() => this.setState({ count: this.state.count + 1 })}>
          Click me
        </button>
      </div>
    );
  }
}
```



# React Component API

- React component is a top-level API. It makes the code completely individual and reusable in the application.
- It includes various methods for **Creating elements**  
**Transforming elements and Fragments**
- Three most important methods available in the React component API.
  1. **setState()**
  2. **forceUpdate()**
  3. **findDOMNode()** -  
ReactDOM.findDOMNode(component);

# React Components Lifecycle

- Each component in React has a lifecycle which you can monitor and manipulate during its three main phases.
- The three phases are:
  1. **Mounting** – Birth of your component
  2. **Update** – Growth of your component
  3. **Unmount** – Death of your component

# Component Lifecycle Methods

- Mounting (invoked only once)
  - `constructor()`
  - `render()`
  - `componentDidMount()`
- Updating
  - `render()`
  - `componentDidUpdate()`
- Unmounting (invoked only once)
  - `componentWillUnmount()`

# Mounting

- Mounting means putting elements into the DOM.
- React has **four built-in methods that gets called**, in this order, when mounting a component:
  1. **constructor()** – called every time you make a component by React.
  2. **getDerivedStateFromProps()** – The `getDerivedStateFromProps()` method is called right before rendering the element(s) in the DOM.
    - This is the natural place to set the state object based on the initial props.
    - It takes state as an argument, and returns an object with changes to the state.
  3. **render()** - This is the method that actual outputs HTML to the DOM.
  4. **componentDidMount()** - called after the component is rendered.
    - This is where you run statements that requires that the component is already placed in the DOM.
- The **render() method** is required and will always be called, the others are optional and will be called if you define them.

# Updating

- The next phase in the lifecycle is when a component is updated. A component is updated whenever **there is a change in the component's state or props.**
- React has five(5) built-in methods that gets called, in this order, when a component is updated:
  1. `getDerivedStateFromProps()`
  2. `shouldComponentUpdate()`
  3. `render()`
  4. `getSnapshotBeforeUpdate()`
  5. `componentDidUpdate()`
- The `render()` method is required and will always be called, the others are optional and will be called if you define them.

# Unmounting

- The next phase in the lifecycle is when a component is removed from the DOM, or unmounting as React likes to call it.
- React has only one built-in method that gets called when a component is unmounted:
- **componentWillUnmount()** - is called when the component is about to be removed from the DOM.



# React Events

- React can perform actions based on user events Just like HTML.
- React has the same events as HTML: **click, change, mouseover**.
- React events are written in camelCase syntax.
- React event handlers are written inside curly{ } braces.

**<button onClick={print}> Click me!</button>**

```
import React from 'react';
import ReactDOM from 'react-dom';

function shoot() {
  alert("Great Shot!");
}

const myelement = (
  <button onClick={shoot}>Take the shot!</button>
);

ReactDOM.render(myelement, document.getElementById('root'));
```

Take the shot!

An embedded page at cdpn.io says

Great Shot!

OK

# Event handler in Class component

- Place the **sayHello** function inside the app component  
**onClick={this.sayHello}**
- Inline event handlers - Give us lots of new options by using an event handler directly in JSX:

```
class App extends Component {  
  
  constructor(props) {  
    super(props);  
    this.sayHello = this.sayHello.bind(this);  
  }  
  
  sayHello() {  
    alert('Hello!');  
  }  
  
  render() {  
    return (  
      <button onClick={this.sayHello}>  
        Click me!  
      </button>  
    );  
  }  
}  
  
export default App;
```

```
import React from 'react';  
  
function App() {  
  const [count, setCount] = React.useState(0);  
  
  return (  
    <div>  
      Count: {count}  
  
      <button  
        type="button"  
        onClick={function() {  
          setCount(count + 1);  
        }}  
      >
```

# Synthetic Event

- An object that contains useful metadata related to the event, such as the target element's value
- **Example: Pass a Button Value to an Inline Function**
- The value *e* that's returned from the **onClick** event handler:

```
function App() {  
  return (  
    <button value="hello!" onClick={e => alert(e.target.value)}>  
      Click me!  
    </button>  
  );  
}  
  
export default App;
```

# Arrow functions

- Arrow functions are shorthand functions which automatically bind to the surrounding context.
- A shorter syntax than typical JavaScript functions.
- No binding of **this** keyword.

## Normal Function

```
function funcName() {  
  // func body  
}  
funcName()
```

## Arrow function

```
const funcName = () => {  
  // func body  
}  
funcName()
```

```
const square = (x) => x * x;  
console.log(square(21));
```

# Arrow Vs Normal Functions

- Arrow function do not have their own binding of this, so your **this.setState** refer to the **YourClass.setState**.
- Using normal function, you need to bind it to the class to obtain Class's this reference. So when you call **this.setState** actually it refer to **YourFunction.setState()**.

```
class FancyComponent extends Component {  
  handleChange(event) {  
    this.setState({ event }) // `this` is instance of handleChange  
  }  
  
  handleChange = (event) => {  
    this.setState({ event }) // `this` is instance of FancyComponent  
  }  
}
```

# Arrow functions and React Events

- For methods in React, the **this** keyword should represent the component that owns the method.
- That is why we should use arrow functions. With arrow functions, this will always represent the object that defined the arrow function.



# Events - Passing Arguments

Send parameters into an event handler, you have two options

1. Make an anonymous arrow function
2. Bind the event handler to this.

```
class Football extends React.Component {  
  shoot = (a) => {  
    alert(a);  
  }  
  render() {  
    return (  
      <button onClick={() =>  
this.shoot("Goal")}>Take the shot!</button>  
    );  
  }  
}
```

```
ReactDOM.render(<Football />,  
document.getElementById('root'));
```

```
class Football extends React.Component {  
  shoot(a) {  
    alert(a);  
  }  
  render() {  
    return (  
      <button onClick={this.shoot.bind(this,  
"Goal")}>Take the shot!</button>  
    );  
  }  
}
```

```
ReactDOM.render(<Football />,  
document.getElementById('root'));
```

# React Forms

- React offers a stateful, reactive approach to build a form to interact with user.
- Rather than the DOM, **Component** usually handles the **React form**.
- Form is usually implemented by using **components**.





# Forms Are Special ...

```
this.state = { name: "John" };
```

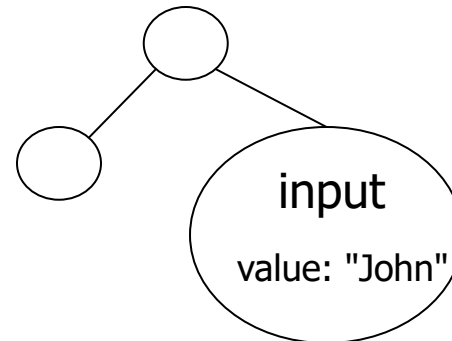
```
<input type="text" value={this.state.name} />
```

**Form**  
Name:

## React

```
state: { name: "John" }
```

## DOM



## ... Forms Are Special

- Form elements keep their internal states in DOM (e.g. `value` for `<input>`)
- The data kind of goes from view → state instead of the state → view
- React only allows *one-way binding*

# Two Ways to Deal with Forms

- **Uncontrolled Components**
  - Use ref to reference DOM elements and get their values
- **Controlled Components (recommended)**
  - Handle `onChange` event
  - Set state in event handler

```
<input type="text"  
  onChange= { event => this.setState(  
    {name: event.target.value}) } />
```

# Uncontrolled component

- **DOM itself handles** the form data.
- HTML elements **maintain their own state** that will be updated when the input value changes.
- Need to use a **ref** to the input field value from the DOM.



A screenshot of a web browser window at localhost:8080. The page title is "Uncontrolled Form Example". Below the title, there is a form with two text input fields labeled "Name:" and "CompanyName:", followed by a "Submit" button. The inputs are empty.

```
import React, { Component } from 'react';
class App extends React.Component {
  constructor(props) {
    super(props);
    this.updateSubmit = this.updateSubmit.bind(this);
    this.input = React.createRef();
  }
  updateSubmit(event) {
    alert('You have entered the UserName and CompanyName successfully.');
```

```
    event.preventDefault();
  }
}
export default App;

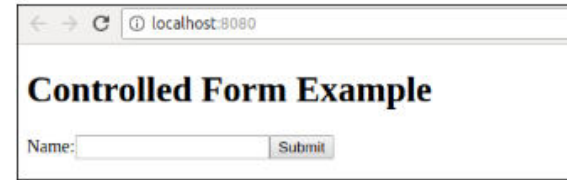
render() {
  return (
    <form onSubmit={this.updateSubmit}>
      <h1>Uncontrolled Form Example</h1>
      <label>Name:
        <input type="text" ref={this.input} />
      </label>
      <label>
        CompanyName:
        <input type="text" ref={this.input} />
      </label>
      <input type="submit" value="Submit" />
    </form>
  );
}
```

# Controlled Component

- Form data is usually handled by the components.
- All the data is **stored in the component state** and will be updated only with **setState()** method.
- You can control changes by adding event handlers in the onChange attribute.
- A controlled component takes its current value through **props** and notifies the changes through **callbacks** like an onChange event.

# Controlled Component - Forms

```
import React, { Component } from 'react';
class App extends React.Component {
  constructor(props) {
    super(props);
    this.state = {value: ""};
    this.handleChange = this.handleChange.bind(this);
    this.handleSubmit = this.handleSubmit.bind(this);
  }
  handleChange(event) {
    this.setState({value: event.target.value});
  }
  handleSubmit(event) {
    alert('You have submitted the input successfully: ' +
    this.state.value);
    event.preventDefault();
  }
}
```

A screenshot of a web browser window with the address bar showing 'localhost:8080'. The page title is 'Controlled Form Example'. Below the title, there is a label 'Name:' followed by a text input field and a 'Submit' button.

```
render() {
  return (
    <form onSubmit={this.handleSubmit}>
      <h1>Controlled Form Example</h1>
      <label>
        Name:
        <input type="text" value={this.state.value} on
Change={this.handleChange} />
      </label>
      <input type="submit" value="Submit" />
    </form>
  );
}
export default App;
```

# React Conditional Rendering

- We can create multiple components which encapsulate behavior that we need.
- After that, we can **render** them depending **on some conditions** or the **state** of our application.
- There is more than one way to do conditional rendering in React
  - if
  - ternary operator
  - logical && operator
  - switch case operator
  - Conditional Rendering with enums

# Conditional Rendering - Example

```
import React, { Component } from 'react';
// Message Component
function Message(props)
{
  if (props.isLoggedIn)
    return <h1>Welcome Back!!!</h1>;
  else
    return <h1>Please Login First!!!</h1>;
}
// Login Component
function Login(props)
{ return(
  <button onClick = {props.clickInfo}> Login </button>
); }
// Logout Component
function Logout(props)
{
  return(
    <button onClick = {props.clickInfo}> Logout </button>
  ); }
}
```

```
class App extends Component{
  constructor(props)
  {
    super(props);
    this.handleLogin = this.handleLogin.bind(this);
    this.handleLogout = this.handleLogout.bind(this);
    this.state = {isLoggedIn : false};
  }
  handleLogin()
  { this.setState({isLoggedIn : true});}
  handleLogout()
  { this.setState({isLoggedIn : false});}
  render(){
    return(
      <div>
        <h1> Conditional Rendering Example </h1>
        <Message isLoggedIn = {this.state.isLoggedIn}/> {
          (this.state.isLoggedIn)?(
            <Logout clickInfo = {this.handleLogout} />
          ) : (
            <Login clickInfo = {this.handleLogin} /> ) }
        </div> ); } }
  export default App;
```





# Output

When you execute the above code, you will get the following screen.



After clicking the login button, you will get the below screen.

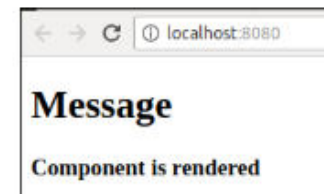


# Preventing Component from Rendering

- Sometimes it might happen that a component hides itself even though another component rendered it.
- To do this (prevent a component from rendering), we will have to return **null** instead of its render output

```
import React from 'react';
import ReactDOM from 'react-dom';
function Show(props)
{
  if(!props.displayMessage)
    return null;
  else
    return <h3>Component is rendered</h3>;
}
ReactDOM.render(
  <div>
    <h1>Message</h1>
    <Show displayMessage = {true} />
  </div>,
  document.getElementById('app')
);
```

Output:



# React Lists

- The **map()** function is used for traversing the lists.
- In the below example, the map() function takes an array of numbers and multiply their values with 5
- We assign the new array returned by map() to the variable multiplyNums and log it.

```
var numbers = [1, 2, 3, 4, 5];  
const multiplyNums = numbers.map((number)=>{  
  return (number * 5);  
});  
console.log(multiplyNums);
```

# React Keys

- Keys React are utilized to identify specific Virtual DOM Elements that have changed.

```
const FruitList = (props) =>{
  const fruits = props.fruits.map((fruit) =>
    <li>{fruit.name}</li>
  )
  return(
    <ul>
      {fruits}
    </ul>
  )
}
```

```
const FruitList = (props) =>{
  const fruits = props.fruits.map((fruit) =>
    <li key={fruit.id}>{fruit.name}</li>
  )
  return(
    <ul>
      {fruits}
    </ul>
  )
}
```

# React Fragments

- Fragments allow you to group a list of children without adding extra nodes to the DOM.
- The render method will require a '**div**' tag and put the entire content or elements inside it.
- It makes the execution of code faster as compared to the div tag and takes less memory.

```
// Rendering with div tag
class App extends React.Component {
  render() {
    return (
      //Extraneous div element
      <div>
        <h2> Hello World! </h2>
        <p> Welcome to the JavaTpoint. </p>
      </div>
    );
  }
}
```

```
// Rendering with fragments tag
class App extends React.Component {
  render() {
    return (
      <React.Fragment>
        <h2> Hello World! </h2>
        <p> Welcome to the JavaTpoint. </p>
      </React.Fragment>
    );
  }
}
```

# The Need for Routing

- Usually we have different pages in a web application
- How do we have different "pages" in a SPA?

[Home](#)   [About](#)   [Login](#)

---

---

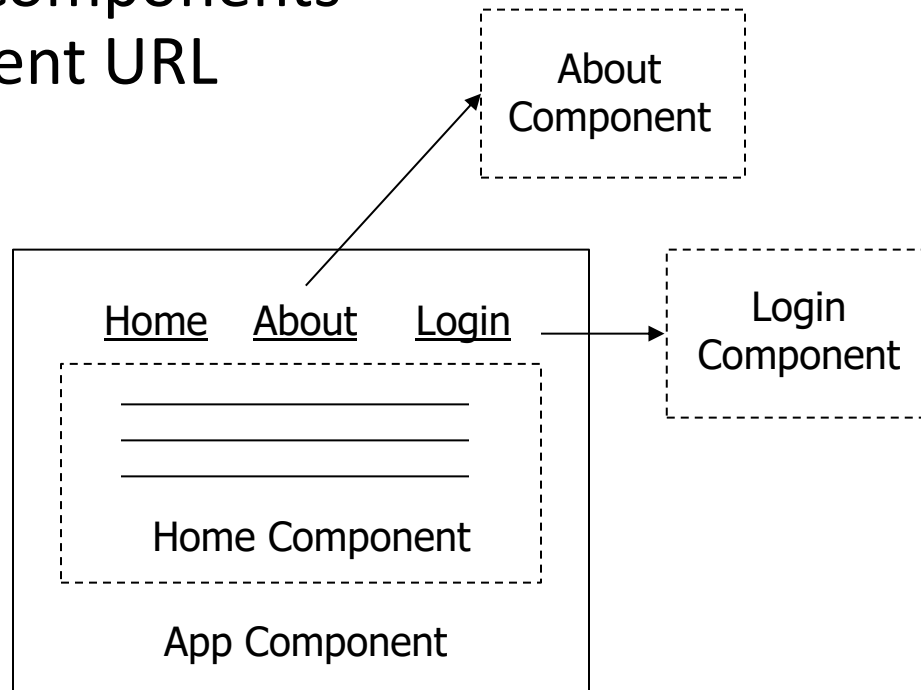
---

---

---

# Routing

- Load different components based on different URL



# React Router

- Routing is a process in which a user is directed to different pages based on their action or request.
- **React Router** is the standard **routing** library for **React**. It keeps your UI in sync with the URL.
- When a user types a specific URL into the browser, and if this URL path matches any 'route' inside the router file, the user will be redirected to that particular route.
- Need to install react router by using this command:

```
npm install react-router-dom --save
```





# Basic Usage of React Router ...

```
<Router>  
  <React.Fragment>  
    <ul>  
      <li><Link to="/">Home</Link></li>  
      <li><Link to="/about">About</Link></li>  
    </ul>  
    <Routes>  
      <Route path="/" exact element={<Home />}> </Route>  
      <Route path="/about" element={< About />}> </Route>  
    </Routes>  
  </React.Fragment>  
</Router>
```

# ... Basic Usage of React Router

- `<Router>` aliased as `BrowserRouter` component listens to URL changes and loads/unloads components. **BrowserRouter** wraps all your `Route` components.
- `<Route>` maps a path to a component
  - `exact` matches the full path instead of just the prefix
  - responsible for showing - or hiding - the components they contain.
- `<Link>` replaces `<a>` so the requests do not go back to the server
  - as you can imagine - used to generate links to your routes

# BrowserRouter

- Import **BrowserRouter** from react-router-dom, and you use it to wrap all your app:
- A BrowserRouter component can only have one child element, so we wrap all we're going to add in a div element.

```
import React from 'react'
import ReactDOM from 'react-dom'
import { BrowserRouter as Router } from 'react-router-dom'

ReactDOM.render(
  <Router>
    <div>
      <!-- -->
    </div>
  </Router>,
  document.getElementById('app')
)
```

# Link

- The Link component is used to trigger new routes.
- Import it from react-router-dom, and you can add the Link components to point at different routes, with the to attribute.

```
import React from 'react'
import ReactDOM from 'react-dom'
import { BrowserRouter as Router, Link } from 'react-router-dom'

ReactDOM.render(
  <Router>
    <div>
      <aside>
        <Link to={` /dashboard`} >Dashboard</Link>
        <Link to={` /about`} >About</Link>
      </aside>
      <!-- -->
    </div>
  </Router>,
  document.getElementById('app')
)
```

# Router

- Now let's add the Route component in the above snippet to make things actually work as we want:

```
import React from 'react'
import ReactDOM from 'react-dom'
import { BrowserRouter as Router, Link, Route } from 'react-router-dom'

const Dashboard = () => (
  <div>
    <h2>Dashboard</h2>
    ...
  </div>
)

const About = () => (
  <div>
    <h2>About</h2>
    ...
  </div>
)
```

```
ReactDOM.render(
  <Router>
    <div>
      <aside>
        <Link to={` /`} >Dashboard</Link>
        <Link to={` /about`} >About</Link>
      </aside>

      <main>
        <Route exact path="/" component={Dashboard} />
        <Route path="/about" component={About} />
      </main>
    </div>
  </Router>,
  document.getElementById('app')
)
```

# React Router Switch

- The **<Switch>** component is used to render components only when the path will be **matched**. Otherwise, it returns to the **not found** component.

```
function App() {  
  return (  
    <main>  
      <Switch>  
        <Route path="/" component={Home} exact />  
        <Route path="/about" component={About} />  
        <Route path="/shop" component={Shop} />  
      </Switch>  
    </main>  
  )  
}
```

# Other React Router Components

- `<BrowserRouter>` vs `<HashRouter>`
  - `BrowserRouter` relies on HTML 5 History API which may not be available on older browsers
  - `BrowserRouter` also relies on the web server configured to serve `index.html` regardless of the path
  - `HashRouter` has `#` in URLs which is aesthetically unpleasing
- `<Link>` vs `<NavLink>`
  - `NavLink` allow more styling (e.g. activated link)
- `<Switch>` renders the first matched `<Route>` in its children

# Components in React Router

- There are two types of router components:  
**<BrowserRouter>**: It is used for handling the dynamic URL.  
**<HashRouter>**: It is used for handling the static request.
- **<BrowserRouter>** builds the classic URL as  
`https://application.com/dashboard /* BrowserRouter */`
- **<HashRouter>** builds the URL with Hash  
`https://application.com/#/dashboard /* HashRouter */`



# Styling React Using CSS

```
import React from 'react';
import ReactDOM from 'react-dom';

class MyHeader extends React.Component {
  render() {
    return (
      <div>
        <h1 style={{color: "red"}}>Hello Style!</h1>
        <p>Add a little style!</p>
      </div>
    );
  }
}
```

**Hello Style!**

Add a little style!

```
class MyHeader extends React.Component {
  render() {
    const mystyle = {
      color: "white",
      backgroundColor: "DodgerBlue",
      padding: "10px",
      fontFamily: "Arial"
    };
    return (
      <div>
        <h1 style={mystyle}>Hello Style!</h1>
        <p>Add a little style!</p>
      </div>
    );
  }
}
```

**Hello Style!**

Add a little style!



# Example

Contact List -- Display Contacts from REST API



# Example – Setting up bootstrap

```
// public/index.html ...
```

```
<head> ...
```

```
<link rel="stylesheet"
```

```
href="https://stackpath.bootstrapcdn.com/bootstrap/4.1.3/css/bootstrap.min.css" integrity="sha384-MCw98/SFnGE8fJT3GXwEOngsV7Zt27NXFoaoApmYm81iuXoPkFOJwJ8ERdknLPMO" crossorigin="anonymous"> ...
```

```
</head>
```

# Example

```
// src/components/contacts.js

import React from 'react'

const Contacts = ({ contacts }) => {
  return (
    <div>
      <center><h1>Contact List</h1></center>
      {contacts.map((contact) => (
        <div class="card">
          <div class="card-body">
            <h5 class="card-title">{contact.name}</h5>
            <h6 class="card-subtitle mb-2 text-muted">{contact.email}</h6>
            <p class="card-text">{contact.company.catchPhrase}</p>
          </div>
        </div>
      ))}
    </div>
  )
};

export default Contacts
```

# Guest Book Example

| My Guest Book               |                                   |                                               |
|-----------------------------|-----------------------------------|-----------------------------------------------|
| John says:                  | Hello!                            | <a href="#">Edit</a>   <a href="#">Delete</a> |
| Jane says:                  | Your website looks nice.          | <a href="#">Edit</a>   <a href="#">Delete</a> |
| Joe says:                   | Nice to meet you. I'm from China. | <a href="#">Edit</a>   <a href="#">Delete</a> |
| <a href="#">Add Comment</a> |                                   |                                               |

| Add Comment                        |                      |
|------------------------------------|----------------------|
| Your name:                         | <input type="text"/> |
| <input type="text"/>               |                      |
| <input type="button" value="Add"/> |                      |

| Edit Entry                                            |                                   |
|-------------------------------------------------------|-----------------------------------|
| Your name:                                            | <input type="text" value="Jane"/> |
| <input type="text" value="Your website looks nice."/> |                                   |
| <input type="button" value="Save"/>                   |                                   |

# Load Data from Server

- When?
- How?

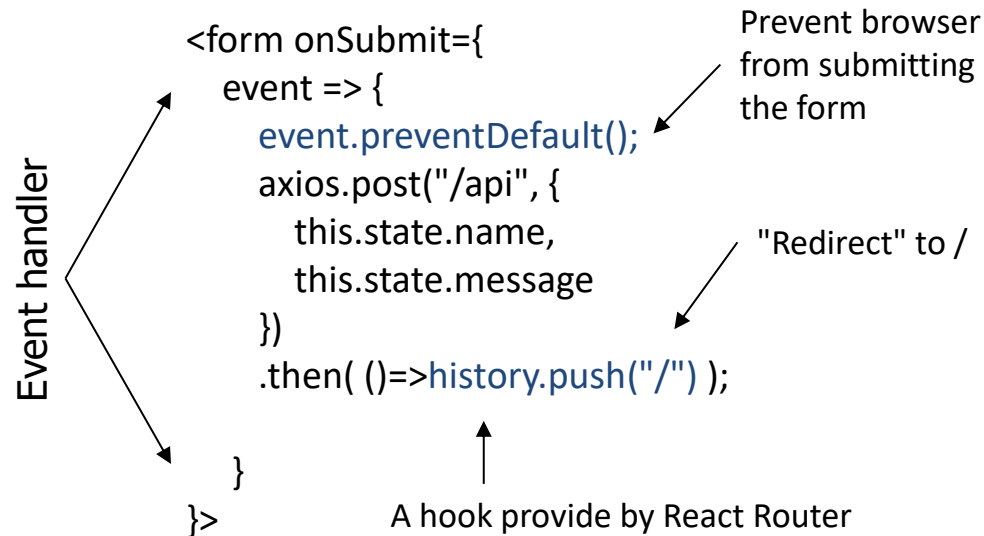


# Fetch vs Axios

- Both are Promise-based
- Fetch is a standard Web API
  - May not be available on older browsers
  - Available for React Native
- Axios is a popular JavaScript HTTP client library
  - Treat error status codes as error so it can be handled in `catch()` instead of `then()`

# Submit a Form

- Handle the `onSubmit` event of the form, e.g.,



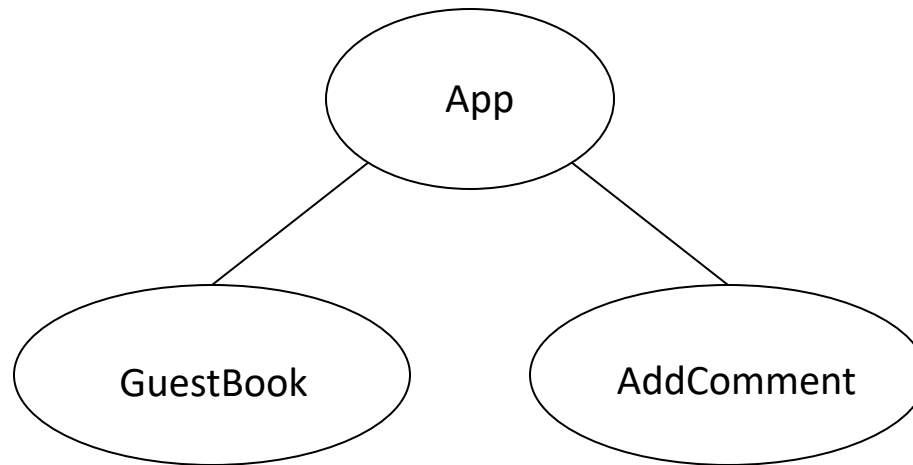


# Keeping Data in SPA ...

- It's inefficient to query the server every time the GuestBook component is mounted
- Maybe we want to keep a local copy of GuestBook in the SPA (the data may get stale but it's more efficient)

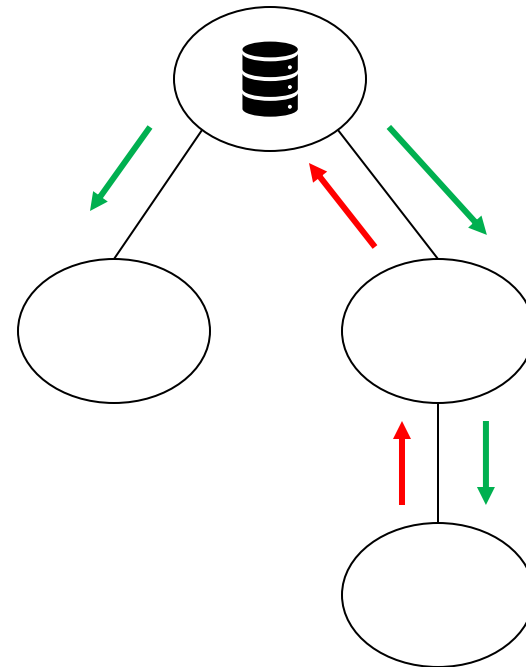
# ... Keeping Data in SPA

- Where the data should be stored?

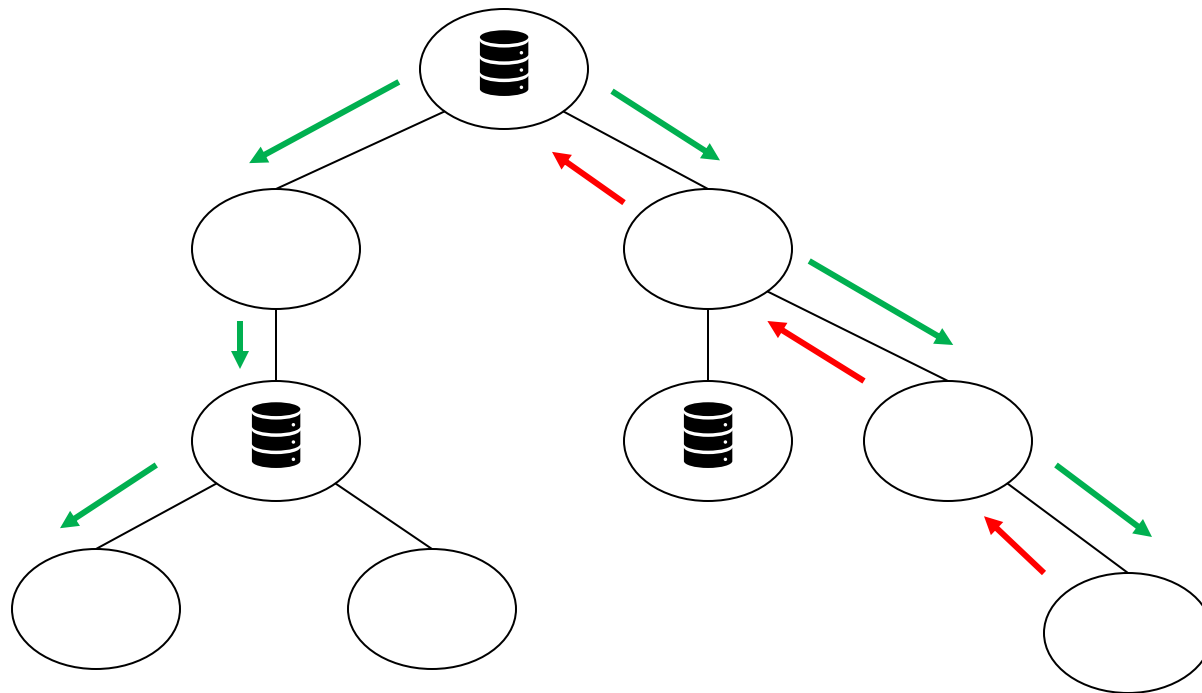


# Basic State Management in React

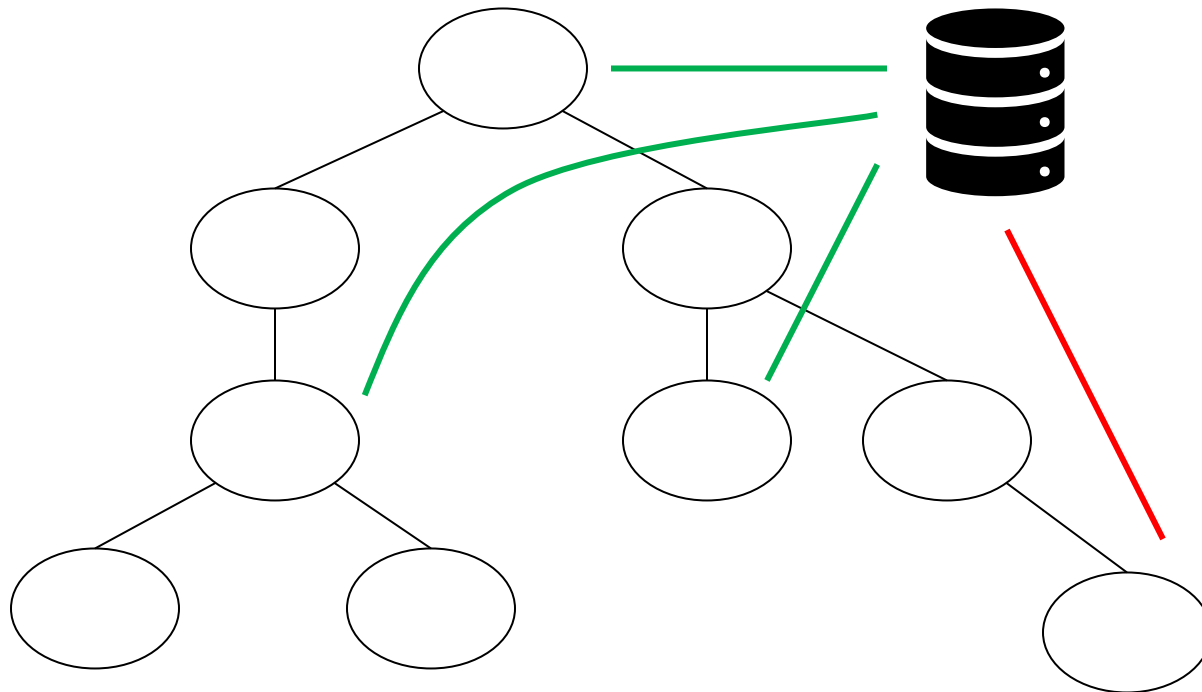
- States stored in *root component*
- Parent passes data down to children as *properties*
- Parent also passes callback functions to children as properties, and children can use those callbacks to send changes back up



# State Management Can Get Difficult and Messy



# Store States in One Place



# State Management Solutions

- React Context API
- Redux
  - More mature and performant
  - More features and tool support

# State Management

## React Context-Example

```
class App extends React.Component {
  render() {
    return <Toolbar theme="dark" />;
  }
}

function Toolbar(props) {
  // The Toolbar component must take an extra "theme" prop
  // and pass it to the ThemedButton. This can become painful
  // if every single button in the app needs to know the theme
  // because it would have to be passed through all components.
  return (
    <div>
      <ThemedButton theme={props.theme} />
    </div>
  );
}

class ThemedButton extends React.Component {
  render() {
    return <Button theme={this.props.theme} />;
  }
}
```

```
// Context lets us pass a value deep into the component tree
// without explicitly threading it through every component.
// Create a context for the current theme (with "light" as the default).
const ThemeContext = React.createContext('light');
```

```
class App extends React.Component {
  render() {
    // Use a Provider to pass the current theme to the tree below.
    // Any component can read it, no matter how deep it is.
    // In this example, we're passing "dark" as the current value.
    return (
      <ThemeContext.Provider value="dark">
        <Toolbar />
      </ThemeContext.Provider>
    );
  }
}
```

```
// A component in the middle doesn't have to
// pass the theme down explicitly anymore.
```

```
function Toolbar() {
  return (
    <div>
      <ThemedButton />
    </div>
  );
}
```

```
class ThemedButton extends React.Component {
  // Assign a contextType to read the current theme context.
  // React will find the closest theme Provider above and use its value.
  // In this example, the current theme is "dark".
  static contextType = ThemeContext;
  render() {
    return <Button theme={this.context} />;
  }
}
```

# Call A RESTful API in React

```
import React, { Component } from 'react'

class App extends Component {
  ...
  componentDidMount() {
    fetch('http://jsonplaceholder.typicode.com/users')
      .then(res => res.json())
      .then((data) => {
        this.setState({ contacts: data })
      })
      .catch(console.log)
  }
  ...
}
```

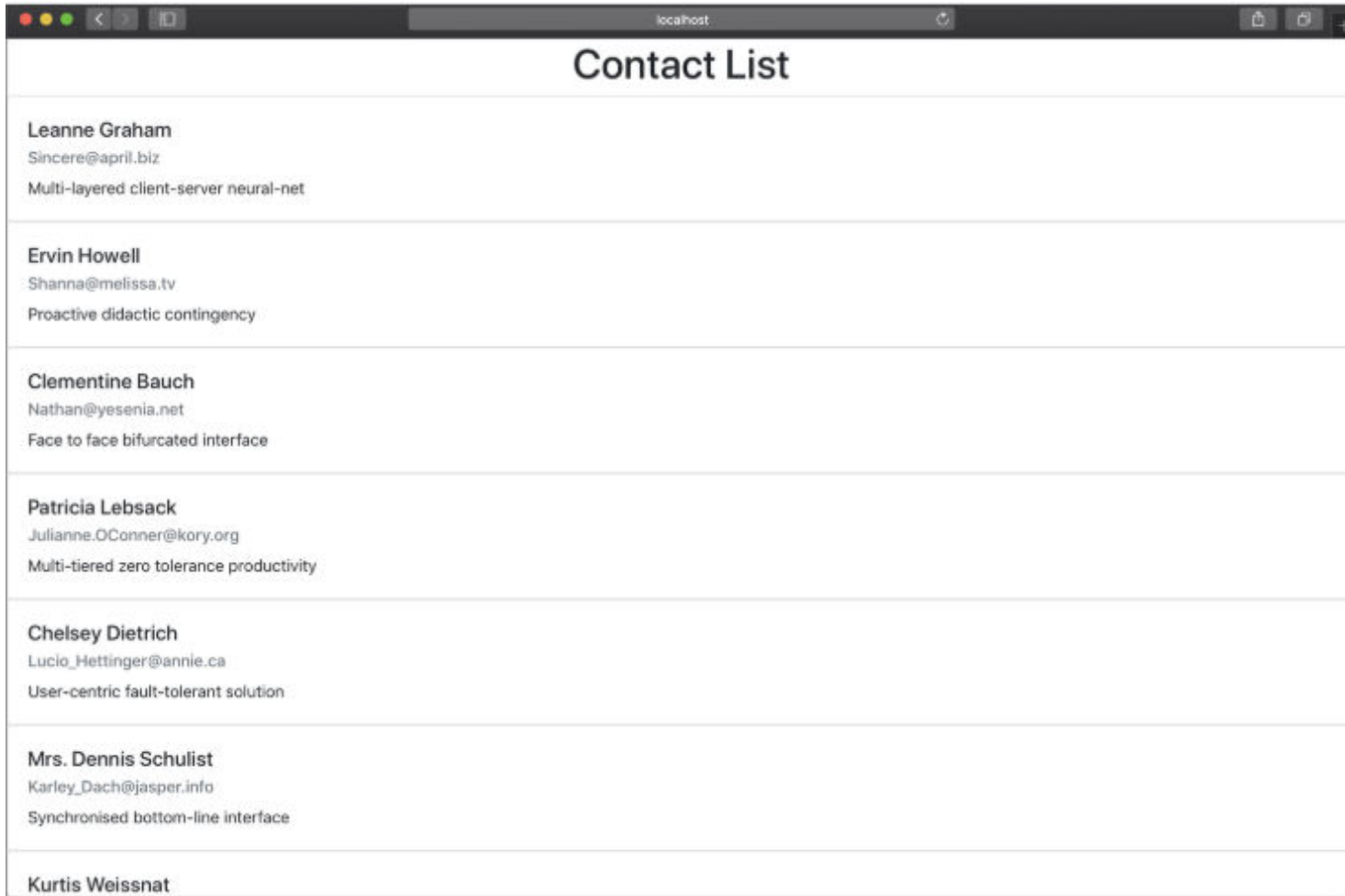
`fetch('http://jsonplaceholder.typicode.com/users')` will make a GET request to the endpoint  
`.then(res => res.json())` parses the output to JSON,  
`.then((data) => {this.setState({ contacts: data })})` sets the value of our state to the output from the API call and finally `.catch(console.log)` logs any error we get to the console.



# Example – App.js

```
class App extends Component {
  state = {
    contacts: []
  }
  componentDidMount() {
    fetch('http://jsonplaceholder.typicode.com/users')
      .then(res => res.json())
      .then((data) => {
        this.setState({ contacts: data })
      })
      .catch(console.log)
  }
  render() {
    return (
      <Contacts contacts={this.state.contacts} />
    );
  }
}
export default App;
```

# Example - Output



A screenshot of a web browser window displaying a 'Contact List' table. The browser's address bar shows 'localhost'. The table has 7 rows, each representing a contact with their name, email address, and a brief description of their work or project.

| Contact List                |                           |                                          |
|-----------------------------|---------------------------|------------------------------------------|
| <b>Leanne Graham</b>        | Sincere@april.biz         | Multi-layered client-server neural-net   |
| <b>Ervin Howell</b>         | Shanna@melissa.tv         | Proactive didactic contingency           |
| <b>Clementine Bauch</b>     | Nathan@yesenia.net        | Face to face bifurcated interface        |
| <b>Patricia Lebsack</b>     | Julianne.OConner@kory.org | Multi-tiered zero tolerance productivity |
| <b>Chelsey Dietrich</b>     | Lucio_Hettinger@annie.ca  | User-centric fault-tolerant solution     |
| <b>Mrs. Dennis Schulist</b> | Karley_Dach@jasper.info   | Synchronised bottom-line interface       |
| <b>Kurtis Weissnat</b>      |                           |                                          |

# Some React Resources

- [React Bootstrap](#)
- [React Icons](#)
- [Awesome React Components](#)
- [bit.dev](#) for publishing, sharing, and searching components



# Readings

- [React](#)
- [React Router](#)
- [Redux](#) and [React Redux](#)



Thank you!



National University  
Of Computer and Emerging Sciences